



Diseño conceptual y lógico de la base de datos

Andres Camilo Meneses Ortega
David Hernando Monsalve Delima
Eduardo Trujillo Santos
Nicolás Cifuentes Barriga

Maestría en Arquitectura de Software, Universidad de la Sabana

Diseño y Optimización de Bases de Datos MAS 2026-3 G1G2

Miguel Alfonso Varela

2026

1. Resumen

La plataforma Ecommify representa un entorno de comercio electrónico multivendedor que requiere gestionar eficientemente información transaccional, analítica y semiestructurada asociada a pedidos, pagos, clientes, productos, reseñas y comportamiento de usuarios. A partir del análisis exploratorio realizado sobre el dataset Brazilian E-commerce (Olist), se identificaron relaciones altamente conectadas entre entidades, presencia de atributos opcionales, crecimiento temporal de operaciones y distribución geográfica concentrada en regiones urbanas, evidenciando necesidades diferenciadas de almacenamiento y procesamiento de datos.

Con base en estos hallazgos, se seleccionó una arquitectura políglota híbrida compuesta por PostgreSQL y MongoDB. PostgreSQL fue definido como sistema transaccional principal para la gestión de órdenes, pagos, clientes, vendedores e inventario, garantizando integridad referencial, propiedades ACID y consistencia fuerte mediante esquemas normalizados en tercera forma normal, particionamiento temporal y soporte geoespacial con PostGIS. Por su parte, MongoDB fue seleccionado para manejar catálogos de productos, reseñas y carritos de compra debido a la naturaleza flexible y semiestructurada de estos datos, permitiendo desnormalización eficiente, alta disponibilidad y escalabilidad horizontal.

La propuesta incorpora estrategias de sincronización mediante Change Data Capture (CDC), optimización analítica y separación entre cargas OLTP y OLAP, permitiendo construir una solución moderna, escalable y alineada con los requerimientos funcionales y no funcionales de plataformas e-commerce de alto volumen.

2. Análisis de requisitos

El análisis de requisitos permitió identificar necesidades diferenciadas de consistencia, disponibilidad y flexibilidad de esquema dentro de la plataforma Ecommify. Los módulos asociados a órdenes, pagos e inventario requieren integridad referencial estricta y cumplimiento de propiedades ACID, motivo por el cual fueron asignados a PostgreSQL bajo un modelo relacional normalizado. Por otro lado, entidades como productos, reseñas y carritos presentan atributos dinámicos, estructuras semiestructuradas y patrones de lectura intensiva, haciendo más apropiado el uso de MongoDB como motor documental flexible y escalable.

Adicionalmente, los altos volúmenes de datos temporales y geográficos identificados durante el análisis exploratorio justifican el uso de estrategias de particionamiento, indexación temporal y capacidades geoespaciales mediante PostGIS, así como procesos de desnormalización analítica para optimizar consultas OLAP y dashboards operacionales.

2.1 Contexto del sistema

Ecommify es una plataforma de comercio electrónico multivendedor especializada en productos tecnológicos, diseñada para permitir la interacción entre clientes, vendedores y operadores logísticos dentro de un ecosistema digital de alta concurrencia. La plataforma debe soportar procesos críticos asociados a gestión de pedidos, pagos, inventario, catálogos de productos, reseñas y análisis de comportamiento de usuarios.

A partir del análisis exploratorio realizado sobre el dataset Brazilian E-commerce (Olist), se identificó una estructura de datos altamente conectada y heterogénea, compuesta por entidades transaccionales y analíticas que presentan diferentes patrones de acceso, volumen y consistencia. Mientras los procesos relacionados con órdenes, pagos e inventario requieren integridad referencial estricta y cumplimiento de propiedades ACID, otros módulos como catálogo de productos, reviews y sesiones de usuario demandan flexibilidad de esquema, alta disponibilidad y capacidad de escalabilidad horizontal.

En consecuencia, se seleccionó una arquitectura políglota híbrida basada en PostgreSQL y MongoDB, permitiendo utilizar tecnologías especializadas según la naturaleza de cada conjunto de datos y los requerimientos operacionales del sistema.

2.2 Requisitos

2.2.1 Requisitos Funcionales

ID	Requisito funcional
RF-01	Registrar clientes y gestionar información de usuarios.
RF-02	Permitir la creación y seguimiento de pedidos.
RF-03	Gestionar múltiples ítems por pedido.

ID	Requisito funcional
RF-04	Procesar pagos con diferentes métodos de pago.
RF-05	Administrar vendedores y productos asociados.
RF-06	Gestionar inventario y disponibilidad de productos.
RF-07	Permitir almacenamiento de reseñas y comentarios de clientes.
RF-08	Gestionar catálogos de productos con atributos variables por categoría.
RF-09	Permitir análisis geográficos y temporales de pedidos.
RF-10	Gestionar carritos de compra temporales y sesiones de usuario.
RF-11	Soportar consultas analíticas y dashboards operacionales.
RF-12	Permitir integración y sincronización entre PostgreSQL y MongoDB.

Tabla 1. Requisitos funcionales.

2.2.2 Requisitos no Funcionales

ID	Requisito funcional
RNF-01	Garantizar integridad referencial en operaciones transaccionales.
RNF-02	Mantener tiempos de respuesta inferiores a 100 ms para operaciones OLTP.
RNF-03	Soportar más de 100.000 pedidos y transacciones.
RNF-04	Permitir escalabilidad horizontal en módulos analíticos y documentales.
RNF-05	Garantizar disponibilidad superior al 99% en ambientes de prueba.
RNF-06	Optimizar consultas geográficas y temporales.
RNF-07	Permitir evolución flexible de esquemas para productos y reviews.
RNF-08	Separar cargas transaccionales (OLTP) y analíticas (OLAP).
RNF-09	Permitir particionamiento temporal y optimización de almacenamiento.
RNF-10	Garantizar seguridad y consistencia en información financiera.

Tabla 2. Requisitos no funcionales.

2.2.3 Identificación de entidades del dominio

Entidad	Descripción	Tecnología
Customers	Información de clientes y compradores.	PostgreSQL
Orders	Órdenes realizadas por clientes.	PostgreSQL
Order Items	Productos incluidos en cada pedido.	PostgreSQL
Payments	Información financiera y pagos asociados a órdenes.	PostgreSQL
Sellers	Vendedores registrados en la plataforma.	PostgreSQL
Inventory	Gestión de stock y disponibilidad.	PostgreSQL
Geolocations	Información geoespacial y coordenadas.	PostgreSQL + PostGIS
Products	Catálogo flexible de productos tecnológicos.	MongoDB
Reviews	Comentarios y calificaciones de usuarios.	MongoDB
Carts	Carritos de compra temporales.	MongoDB
Analytics	Datos desnormalizados para consultas analíticas.	MongoDB / PostgreSQL réplica

Tabla 3. Entidades principales del dominio.

2.2.4 Identificación de entidades del dominio

Entidad	Volumen inicial estimado	Crecimiento esperado	Crecimiento esperado
Customers	100.000	Alto	OLTP
Orders	100.000+	Muy alto	OLTP
Order Items	120.000+	Muy alto	OLTP
Payments	100.000+	Alto	OLTP
Products	50.000+	Alto	Lectura intensiva
Reviews	100.000+	Muy alto	Analítico
Sellers	5.000	Medio	OLTP
Inventory	50.000+	Alto	Transaccional

Entidad	Volumen inicial estimado	Crecimiento esperado	Crecimiento esperado
Geolocations	1.000.000+	Bajo	Analítico
Carts	Variable	Muy alto	Alta concurrencia
Analytics	Millones de registros	Muy alto	OLAP

Tabla 4. Entidades principales del dominio.

3. Diseño conceptual

3.1 Diagrama ER completo de alta calidad

3.2 Descripción detallada de cada entidad y relación

- **Customers:** Representa a los clientes registrados en la plataforma Ecommify y constituye el punto de inicio de las operaciones comerciales dentro del sistema. Su propósito principal es almacenar la información necesaria para identificar usuarios, asociar pedidos y realizar análisis de comportamiento y segmentación geográfica. Cada cliente posee un identificador único (*customer_id*) utilizado como clave primaria y como referencia en las entidades transaccionales. Adicionalmente, se almacena un identificador persistente (*customer_unique_id*) que permite reconocer usuarios recurrentes incluso cuando realizan múltiples órdenes. Los atributos geográficos como ciudad, estado y código postal permiten soportar análisis territoriales, optimización logística y recomendaciones regionales.
Relación con Orders: Un cliente puede realizar múltiples órdenes. Cada orden pertenece exclusivamente a un cliente.
Tipo de relación: Customers (1) ----- (N) Orders.
- **Orders:** Representa el núcleo transaccional del sistema y almacena la información asociada a cada pedido realizado en la plataforma. Esta entidad concentra el flujo principal del negocio, incluyendo estados del pedido, fechas del ciclo de vida de entrega y montos asociados. Las órdenes requieren propiedades ACID debido a que afectan inventario, pagos y trazabilidad comercial. Por esta razón se implementan en PostgreSQL bajo un esquema altamente normalizado. Los atributos temporales permiten realizar análisis históricos, métricas logísticas y monitoreo operacional.
Relación con Customers: Cada orden pertenece a un único cliente.
Relación con Order Items: Una orden puede contener múltiples productos diferentes.
Relación con Payments: Una orden puede dividirse en múltiples pagos o cuotas.
Relación con Reviews: Una orden puede tener asociada una reseña posterior a la entrega.
Tipo de relación: Orders (1) ----- (N) Order_Items. Orders (1) ----- (N) Payments. Orders (1) ----- (N) Reviews.
- **Order_Items:** Representa el detalle individual de productos asociados a una orden. Esta tabla permite modelar pedidos compuestos por múltiples artículos vendidos incluso por distintos vendedores. Se almacena el precio unitario como snapshot histórico para preservar consistencia financiera aun cuando el precio actual del producto cambie posteriormente. También se incluye el costo de envío individual y fechas límite logísticas.
Relación con Orders: Cada ítem pertenece a una única orden.
Relación con Products: Cada ítem referencia un producto específico.
Relación con Sellers: Cada ítem está asociado a un vendedor determinado.

Tipo de relación: Order_Items (N) ----- (1) Orders. Order_Items (N) ----- (1) Products. Order_Items (N) ----- (1) Sellers.

- **Payments:** Almacena la información financiera relacionada con los pagos realizados por los clientes. Esta entidad soporta múltiples métodos de pago y pagos fraccionados asociados a una misma orden.

Debido a la naturaleza variable de algunos métodos de pago, se utiliza un atributo *JSONB* para almacenar metadata adicional como tokens, respuestas de pasarelas de pago o información antifraude.

El manejo de esta entidad requiere alta consistencia e integridad transaccional.

Relación con Orders: Cada pago pertenece a una orden específica.

Tipo de relación: Orders (1) ----- (N) Payments.

- **Sellers:** Representa a los vendedores registrados en el marketplace. Cada vendedor puede comercializar múltiples productos y participar en numerosas órdenes.

La información geográfica asociada permite realizar análisis de distribución comercial, tiempos de entrega y cobertura logística.

Relación con Order_Items: Un vendedor puede aparecer en múltiples ítems de órdenes.

Tipo de relación: Sellers (1) ----- (N) Order_Items.

- **Inventory:** Administra la disponibilidad y control de stock de productos. Su propósito es garantizar trazabilidad y consistencia en las operaciones relacionadas con existencias, reservas y procesos logísticos.

La actualización de inventario requiere consistencia inmediata para evitar sobreventa o inconsistencias en pedidos concurrentes.

Relación con Products: Cada producto posee un registro de inventario asociado.

Tipo de relación: Products (1) ----- (1) Inventory.

- **Geolocations:** Almacena coordenadas geográficas asociadas a códigos postales, ciudades y estados. Esta información permite ejecutar consultas espaciales y análisis geográficos avanzados mediante la extensión PostGIS de PostgreSQL.

El modelo soporta cálculos de distancia, clustering territorial y optimización logística.

Relación con Customers y Sellers: La información geográfica se utiliza como referencia territorial para clientes y vendedores.

Tipo de relación: Geolocations (1) ----- (N) Customers. Geolocations (1) ----- (N) Sellers.

- **Products (MongoDB):** Corresponde al catálogo flexible de productos tecnológicos y fue modelada en MongoDB debido a la variabilidad estructural de atributos entre categorías. Cada producto puede contener especificaciones dinámicas diferentes dependiendo de su naturaleza, tales como características técnicas, dimensiones, etiquetas, multimedia o atributos personalizados.

MongoDB permite almacenar esta información sin necesidad de alterar esquemas relacionales ni generar grandes cantidades de columnas nulas.

Relación con Order_Items: Los productos pueden aparecer en múltiples órdenes.

Relación con Inventory: Cada producto posee información de inventario asociada.

Tipo de relación: Products (1) ----- (N) Order_Items. Products (1) ----- (1) Inventory.

- **Reviews (MongoDB):** Almacena comentarios, calificaciones y retroalimentación de clientes sobre productos y órdenes. Debido a la naturaleza semiestructurada de los

comentarios y posibles extensiones futuras como imágenes, etiquetas o análisis de sentimiento, se implementa en MongoDB.

Esta entidad soporta cargas analíticas intensivas y modelos de recomendación.

Relación con Orders: Una review pertenece a una orden específica.

Relación con Customers: Un cliente puede generar múltiples reseñas.

Tipo de relación: Orders (1) ----- (N) Reviews. Customers (1) ----- (N) Reviews.

- **Carts (MongoDB):** Representa los carritos temporales de compra utilizados por los clientes antes de confirmar una orden. Se modela en MongoDB debido a su naturaleza volátil, alta concurrencia y necesidad de expiración automática.

La estructura documental permite almacenar listas dinámicas de productos y snapshots temporales de precios.

Relación con Customers: Cada carrito pertenece a un cliente.

Tipo de relación: Customers (1) ----- (1) Carts.

3.3 Restricciones de negocio identificadas

3.3.1 Restricciones transaccionales

- Una orden debe pertenecer obligatoriamente a un único cliente válido.
- Un pedido debe contener al menos un ítem asociado.
- Los pagos no pueden existir sin una orden relacionada.
- El valor total pagado debe corresponder al monto total de la orden.
- El inventario no puede tomar valores negativos.
- No se puede confirmar una orden si el stock disponible es insuficiente.
- El estado de una orden debe pertenecer a un conjunto controlado de estados válidos: pending, approved, shipped, delivered y canceled.

3.3.2 Restricciones de integridad referencial

- Todo *product_id* en Order_Items debe existir previamente en Products.
- Todo *seller_id* asociado a una orden debe existir en Sellers.
- Toda review debe estar asociada a una orden existente.
- Los pagos deben mantener trazabilidad completa hacia la orden correspondiente.

3.3.3 Restricciones temporales

- La fecha de aprobación de una orden no puede ser anterior a la fecha de compra.
- La fecha de entrega no puede ser anterior a la fecha de despacho.
- Los carritos de compra deben expirar automáticamente después de un periodo determinado de inactividad.

3.3.2 Restricciones analíticas y operacionales

- El sistema debe soportar crecimiento horizontal del catálogo de productos sin necesidad de migraciones frecuentes de esquema.

- El sistema debe permitir consultas geoespaciales optimizadas para análisis logísticos.
- La arquitectura debe separar cargas OLTP y OLAP para evitar degradación del rendimiento transaccional.
- La sincronización entre PostgreSQL y MongoDB debe preservar consistencia eventual en módulos analíticos.
- Los datos históricos de precios en Order_Items no deben modificarse después de creada la orden.
- Las reseñas únicamente podrán generarse sobre órdenes entregadas exitosamente.

4. Diseño lógico - Módulo PostgreSQL

4.1 Esquema relacional normalizado

El diseño lógico relacional de Ecommify fue construido siguiendo principios de normalización hasta tercera forma normal (3FN), con el objetivo de minimizar redundancia, garantizar integridad referencial y optimizar operaciones transaccionales OLTP.

La normalización aplicada asegura que:

- Cada entidad representa un único concepto del dominio.
- Los atributos dependen exclusivamente de la clave primaria.
- No existen dependencias transitivas.
- Las relaciones N:M sean resueltas mediante tablas intermedias.
- Los datos históricos transaccionales permanecen inmutables.

La arquitectura relacional se implementará en PostgreSQL debido a sus capacidades ACID, soporte avanzado de tipos de datos, extensibilidad y funcionalidades analíticas/geoespaciales.

4.2 Esquema relacional final tabla por tabla

Customers: Representa a los clientes registrados en la plataforma. UUID evita colisiones y facilita la escalabilidad distribuida. Separación entre *customer_id* y *customer_unique_id* permite trazabilidad histórica.

Columna	Tipo de dato	Restricciones
customer_id	UUID	PK, NOT NULL
customer_unique_id	UUID	NOT NULL
zip_code_prefix	VARCHAR(10)	NOT NULL
city	VARCHAR(100)	NOT NULL
state	CHAR(2)	NOT NULL
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Tabla 5. Tabla final Customers.

Sellers: Almacena vendedores registrados en el marketplace.

Columna	Tipo de dato	Restricciones
seller_id	UUID	PK, NOT NULL

Columna	Tipo de dato	Restricciones
zip_code_prefix	UUID	NOT NULL
city	VARCHAR(100)	NOT NULL
state	CHAR(2)	NOT NULL
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Tabla 6. Tabla final Sellers.

Orders: Almacena vendedores registrados en el marketplace. El atributo *shipping_address* fue modelado usando *JSONB* debido a variabilidad estructural entre direcciones, flexibilidad para incluir atributos adicionales, reducción de joins, compatibilidad con índices GIN y optimización de consultas documentales ligeras.

Columna	Tipo de dato	Restricciones
order_id	UUID	PK, NOT NULL
customer_id	UUID	FK → customers
order_status	VARCHAR(20)	CHECK
purchase_timestamp	TIMESTAMP	NOT NULL
approved_at	TIMESTAMP	NULL
delivered_carrier_date	TIMESTAMP	NULL
delivered_customer_date	TIMESTAMP	NULL
estimated_delivery_date	TIMESTAMP	NOT NULL
total_amount	NUMERIC(12,2)	CHECK > 0
shipping_address	JSONB	NOT NULL
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Tabla 7. Tabla final Orders.

Order_items: Representa productos asociados a órdenes. El atributo *unit_price* se almacena directamente en la orden para preservar consistencia histórica y evitar alteraciones derivadas de cambios futuros en el catálogo de productos.

Columna	Tipo de dato	Restricciones
order_item_id	BIGSERIAL	PK
order_id	UUID	FK → orders
product_id	UUID	NOT NULL
seller_id	UUID	FK → sellers
quantity	INTEGER	DEFAULT 1
unit_price	NUMERIC(10,2)	CHECK > 0
freight_value	NUMERIC(10,2)	CHECK >= 0
shipping_limit_date	TIMESTAMP	NOT NULL
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Tabla 8. Tabla final Order_items.

Payments: Información financiera asociada a pagos. El campo *payment_metadata* permite almacenar información variable según el método de pago token de transacción, respuesta antifraude, banco emisor, BIN, gateway response y cuotas extendidas.

Columna	Tipo de dato	Restricciones
payment_id	BIGSERIAL	PK
order_id	UUID	FK → orders
payment_sequential	INTEGER	NOT NULL
payment_type	VARCHAR(30)	NOT NULL
payment_installments	INTEGER	CHECK >= 1
payment_value	NUMERIC(12,2)	CHECK > 0
payment_metadata	JSONB	NULL

Columna	Tipo de dato	Restricciones
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Tabla 9. Tabla final Payments.

Inventory: Control de stock y disponibilidad.

Columna	Tipo de dato	Restricciones
inventory_id	BIGSERIAL	PK
product_id	UUID	UNIQUE
stock_quantity	INTEGER	CHECK >= 0
reserved_quantity	INTEGER	CHECK >= 0
warehouse_location	VARCHAR(100)	NOT NULL
updated_at	NUMERIC(12,2)	DEFAULT CURRENT_TIMESTAMP

Tabla 10. Tabla final Inventory.

Geolocations: Información geográfica y espacial. La extensión PostGIS permite consultas espaciales, análisis geográficos, cálculos de distancia, clustering territorial, optimización logística y mapas de calor.

Columna	Tipo de dato	Restricciones
geolocation_id	BIGSERIAL	PK
zip_code_prefix	VARCHAR(10)	NOT NULL
city	VARCHAR(100)	NOT NULL
state	CHAR(2)	NOT NULL
location	GEOGRAPHY(POINT,4326)	NOT NULL

Tabla 11. Tabla final Geolocations.

4.3 Aplicación de tipos avanzados

JSONB: A continuación las utilizaciones de este formato en la base de datos.

Tabla	Campo	Justificación
orders	shipping_address	Flexibilidad documental
payments	payment_metadata	Variabilidad de pagos

Tabla 12. Campos que usan JSONB.

Arrays: Se propone el uso de arrays únicamente en escenarios analíticos o temporales, evitando romper la normalización transaccional. Algunos usos viables son recomendaciones, etiquetas y analítica ligera.

Composite Types: Se propone un tipo compuesto reutilizable para coordenadas geográficas. Con esto se consigue reutilización semántica, consistencia estructural y encapsulamiento geográfico.

4.4 Extensiones PostgreSQL a utilizar

Extensión	Justificación
uuid-oss	Generación de UUIDs distribuidos
pgcrypto	Funciones criptográficas y hashing
postgis	Consultas geoespaciales
btree_gin	Índices híbridos para JSONB
pg_partman	Automatización de particionamiento

Tabla 13. Extensiones postgresql.

4.5 Estrategias de particionamiento

Debido al crecimiento temporal identificado en el análisis exploratorio, se implementará particionamiento sobre entidades de alta volumetría.

Particionamiento de orders: PARTITION BY RANGE (purchase_timestamp). Se utiliza esta partición con el fin de tener consultas temporales frecuentes, crecimiento continuo, optimización de scans, mantenimiento eficiente y archivado histórico.

Partición	Rango
orders_2016	Año 2016
orders_2017	Año 2017
orders_2018	Año 2018

Partición	Rango
orders_current	Datos actuales

Tabla 14. Propuestas de partición para orders.

Particionamiento de order_items: Se propone particionamiento derivado basado en fecha de compra o relación con orders. Con esto se mejoran joins masivos, consultas históricas y cargas OLAP.

Partición	Rango
orders_2016	Año 2016
orders_2017	Año 2017
orders_2018	Año 2018
orders_current	Datos actuales

Tabla 15. Propuestas de partición para order_items.

4.6 Estrategias de indexación

Indices principales

- **Orders:**

INDEX idx_orders_customer(customer_id).

INDEX idx_orders_purchase(purchase_timestamp)

- **Payments:**

INDEX idx_payments_order(order_id)

- **JSONB:**

INDEX idx_payments_order(order_id)

GIN INDEX idx_payments_metadata

- **Geoespacial:**

GIST INDEX idx_geolocation

5. Diseño lógico preliminar – Módulo MongoDB

El módulo MongoDB de Ecommify fue diseñado para gestionar datos semiestructurados, altamente dinámicos y orientados a lectura intensiva, complementando las capacidades transaccionales del módulo PostgreSQL.

A partir del análisis exploratorio del dataset Olist y del estudio de requisitos funcionales y no funcionales, se identificó que ciertas entidades presentan atributos variables, esquemas cambiantes, alta desnormalización analítica y necesidades de escalabilidad horizontal.

MongoDB fue seleccionado debido a su modelo documental flexible, soporte para datos JSON-like (BSON), escalabilidad horizontal, alta disponibilidad y eficiencia en consultas documentales complejas.

El objetivo principal del módulo MongoDB es optimizar catálogos de productos, reseñas, recomendaciones, carritos de compra y cargas analíticas.

5.1 Colecciones principales identificadas

Colección	Propósito
products	Catálogo flexible de productos tecnológicos
reviews	Reseñas y comentarios de usuarios
carts	Carritos de compra temporales
analytics_orders	Datos desnormalizados para consultas analíticas
recommendations	Recomendaciones personalizadas
product_views	Historial de navegación y comportamiento

Tabla 16. Colecciones principales para mongo.

Products: Almacena el catálogo de productos tecnológicos de Ecommify. Debido a que diferentes categorías de productos poseen atributos distintos, se requiere un esquema flexible que permita incorporar propiedades dinámicas sin necesidad de migraciones estructurales frecuentes. MongoDB permite modelar estos productos utilizando documentos heterogéneos y estructuras embebidas.

La colección fue asignada a MongoDB debido a la variabilidad estructural entre categorías, presencia de atributos opcionales, necesidad de alta lectura, soporte multimedia, facilidad de indexación documental y reducción de columnas nulas.

En un modelo relacional, este comportamiento generaría exceso de tablas auxiliares, múltiples joins, columnas vacías y complejidad de mantenimiento.

Reviews: Almacena reseñas, comentarios y calificaciones generadas por clientes después de recibir sus pedidos. Este módulo presenta información semiestructurada que puede evolucionar hacia análisis de sentimiento, multimedia, etiquetas automáticas, moderación y sistemas de recomendación.

La colección fue asignada a MongoDB debido al contenido textual variable, la presencia opcional de multimedia, la estructura extensible, las futuras integraciones IA/NLP y las cargas analíticas intensivas.

Adicionalmente, el análisis exploratorio identificó altos niveles de valores nulos en *review_comment_title* y *review_comment_message* lo cual evidencia comportamiento semiestructurado y flexible.

Carts: Administra carritos temporales de compra antes de la generación definitiva de una orden. Se prioriza la baja latencia, alta concurrencia y expiración automática.

La colección fue asignada a MongoDB debido a la alta frecuencia de actualización, datos temporales, estructura embebida eficiente y necesidad de expiración automática. MongoDB permite evitar joins complejos y manejar sesiones concurrentes de forma eficiente.

Analytics_orders: contiene información desnormalizada orientada a consultas OLAP y dashboards operacionales. Su propósito es desacoplar cargas analíticas del entorno transaccional PostgreSQL.

Esta colección permite dashboards rápidos, agregaciones masivas, análisis históricos, minería de datos, machine learning y reducción de joins complejos.

5.2 Patrones de modelado aplicados

1. **Embedding Pattern:** Se utiliza embedding cuando los datos poseen alta cohesión, baja cardinalidad y acceso conjunto frecuente. Ofreciendo beneficios en reducción de joins, menor latencia, consultas más simples y atomicidad documental. Esto aplica para aspectos como *carts.items* y *products.media*.
2. **Referencing Pattern:** Se utiliza como referencia cuando los datos son altamente reutilizables, poseen crecimiento independiente o alta cardinalidad. Ofreciendo beneficios en reducción de duplicidad, flexibilidad y escalabilidad. Esto aplica para aspectos como *reviews.product_id* y *products.seller_ids*.
3. **Bucket Pattern:** Aplicado en módulos analíticos y eventos temporales. Cuenta con uso potencial para historial de visitas, eventos de navegación y métricas por sesión. Ofreciendo beneficios en reducción de duplicidad, flexibilidad y escalabilidad. Esto aplica para aspectos como *reviews.product_id* y *products.seller_ids*.
4. **Computed Pattern:** Aplicado para almacenar promedios, métricas agregadas, scores y estadísticas precalculadas.

5.3 Justificación de qué datos van a MongoDB vs PostgreSQL

Para realizar esta separación se consideraron criterios de decisión basados en naturaleza del dato, necesidad de consistencia, frecuencia de lectura/escritura, variabilidad estructural, volumetría, escalabilidad y patrón de acceso.

Gracias a esta base la arquitectura híbrida permite mantener consistencia fuerte en procesos críticos, escalar horizontalmente componentes analíticos, optimizar consultas complejas, reducir joins costosos y desacoplar cargas OLTP y OLAP. Esto permite que Ecommify combine la robustez transaccional, flexibilidad documental y capacidad analítica moderna dentro de una arquitectura de datos escalable y preparada para crecimiento futuro.

5.3.1 Datos asignados a PostgreSQL

Entidad	Justificación
customers	Integridad transaccional
orders	ACID y consistencia fuerte
payments	Seguridad financiera
inventory	Control estricto de stock
sellers	Relaciones altamente normalizadas
geolocations	Soporte PostGIS

Tabla 17. Datos asignados a postgresql.

5.3.2 Datos asignados a MongoDB

Colección	Propósito
products	Esquema flexible
reviews	Datos semiestructurados
carts	Alta concurrencia y expiración
analytics_orders	Consultas analíticas rápidas
recommendations	Datos dinámicos y personalizados
product_views	Eventos masivos y escalabilidad

Tabla 18. Datos asignados a mongo.

6. Decisiones arquitectónicas

La arquitectura de datos propuesta para Ecommify fue diseñada bajo un enfoque híbrido políglota, integrando PostgreSQL y MongoDB como motores especializados para diferentes necesidades funcionales y operacionales del sistema.

Las decisiones arquitectónicas fueron tomadas a partir de análisis exploratorio del dataset Brazilian E-commerce (Olist), requerimientos funcionales y no funcionales, comportamiento transaccional y analítico, patrones de acceso, volumetría, consistencia requerida y principios derivados del Teorema CAP.

El objetivo principal de la arquitectura es equilibrar consistencia, disponibilidad, escalabilidad, flexibilidad y rendimiento.

6.1 Matriz de decisión por entidad

Entidad	Tecnología seleccionada	Tipo de carga	Justificación
Customers	PostgreSQL	OLTP	Requiere integridad referencial y consistencia fuerte
Orders	PostgreSQL	OLTP	Entidad transaccional crítica
Order Items	PostgreSQL	OLTP	Relaciones altamente normalizadas
Payments	PostgreSQL	OLTP	Información financiera sensible
Inventory	PostgreSQL	OLTP	Control estricto de stock
Sellers	PostgreSQL	OLTP	Relaciones estructuradas y consistentes
Geolocations	PostgreSQL + PostGIS	Analítica espacial	Consultas geográficas optimizadas
Products	MongoDB	Lectura intensiva	Esquema flexible y atributos variables
Reviews	MongoDB	Analítica / semiestructurada	Comentarios dinámicos y multimedia
Carts	MongoDB	Alta concurrencia	Datos temporales y alta escritura
Analytic Orders	MongoDB	OLAP	Desnormalización y agregaciones rápidas
Recommendations	MongoDB	Personalización	Datos dinámicos y escalables

Entidad	Tecnología seleccionada	Tipo de carga	Justificación
Product Views	MongoDB	Eventos masivos	Alta volumetría y escritura continua

Tabla 19. Matriz de decisión.

La separación entre PostgreSQL y MongoDB responde a la necesidad de optimizar diferentes tipos de carga dentro de Ecommify.

PostgreSQL fue seleccionado para procesos transaccionales críticos, integridad referencial, consistencia fuerte, soporte ACID, relaciones complejas y trazabilidad financiera.

MongoDB fue seleccionado para datos semiestructurados, alta lectura, flexibilidad de esquema, de normalización analítica y escalabilidad horizontal.

Esta combinación permite implementar una arquitectura moderna orientada a microservicios y procesamiento híbrido OLTP/OLAP.

6.2 Análisis del teorema CAP aplicado

El Teorema CAP establece que un sistema distribuido no puede garantizar simultáneamente:

- Consistency (Consistencia).
- Availability (Disponibilidad).
- Partition Tolerance (Tolerancia a particiones).

En ambientes distribuidos modernos, la tolerancia a particiones es obligatoria, por lo que las decisiones arquitectónicas deben balancear consistencia y disponibilidad dependiendo del dominio funcional.

PostgreSQL fue utilizado bajo una orientación de modelo predominante CP debido a que las entidades orders, payments, inventory y customers requieren integridad inmediata, consistencia fuerte, control transaccional y ausencia de anomalías financieras. Por ejemplo, evitar doble pago, evitar sobreventa, mantener trazabilidad de órdenes o garantizar consistencia del inventario.

Esto tiene como implicación que en presencia de fallos distribuidos el sistema puede sacrificar disponibilidad temporal, priorizando integridad y consistencia de datos.

MongoDB fue utilizado bajo una orientación de modelo predominante AP debido a que Las entidades products, reviews, carts y recommendations priorizan disponibilidad, baja latencia, escalabilidad y tolerancia a fallos. Por ejemplo, un carrito puede tolerar consistencia eventual, una recomendación puede estar ligeramente desactualizada o un review puede sincronizarse posteriormente.

Esto tiene como implicación alta disponibilidad, escalabilidad horizontal, distribución geográfica y mejor rendimiento en lectura.

6.3 Trade-offs y estrategias de mitigación

Consistencia y Disponibilidad: La separación entre PostgreSQL y MongoDB introduce consistencia eventual, duplicidad parcial de información y sincronización asíncrona. La mitigación se basa en uso de Change Data Capture (CDC), colas de eventos, sincronización incremental y validaciones periódicas.

Normalización y Rendimiento: La normalización relacional mejora integridad pero aumenta joins, incrementa complejidad analítica y puede afectar rendimiento OLAP. La mitigación se basa en uso de colecciones desnormalizadas, materialización analítica, caching documental y pipelines ETL.

Flexibilidad y Control de esquema: MongoDB ofrece flexibilidad, pero puede generar inconsistencia estructural, dificultad de validación y divergencia documental. La mitigación se basa en la validación de JSON Schema, contratos API, versionamiento de documentos y gobernanza de datos.

Escalabilidad vs Complejidad operacional: Una arquitectura híbrida incrementa la complejidad de despliegue, monitoreo, sincronización y administración. La mitigación se basa en la separación clara de responsabilidades, automatización DevOps, observabilidad centralizada y microservicios desacoplados.

Riesgo	Estrategia
Inconsistencia eventual	CDC + eventos
Alta carga analítica	Colecciones desnormalizadas
Escalabilidad OLTP	Particionamiento PostgreSQL
Consultas geográficas complejas	PostGIS
Cambios frecuentes de esquema	MongoDB flexible
Sobreventa de inventario	Transacciones ACID
Saturación OLAP	Separación OLTP/OLAP

Tabla 20. Estrategias de mitigación implementadas.

6.4 Flujos de sincronización

La arquitectura híbrida requiere sincronización controlada entre PostgreSQL y MongoDB para mantener consistencia analítica y documental. La sincronización será implementada mediante Change Data Capture (CDC), colas de eventos, procesos ETL y replicación parcial.

Flujo de órdenes analíticas: Con el objetivo de permitir dashboards, recomendaciones, métricas, minería de datos y/o machine learning. El flujo planteado es:

Orders (PostgreSQL) → Change Data Capture (CDC) → Event Bus / Kafka → Transformación ETL → analytics_orders (MongoDB)

Flujo de reviews: Con el objetivo de mantener ratings precalculados, recomendaciones rápidas y búsquedas optimizadas. El flujo planteado es:

MongoDB Reviews → Agregaciones → Actualización de ratings → Products Collection

Flujo de inventario: El inventario permanece en PostgreSQL debido a la búsqueda de consistencia fuerte, control concurrente y prevención de sobreventa. El flujo planteado es:

Orders Confirmed → PostgreSQL Transaction → Inventory Update → Sync Event → MongoDB Analytics

Tipo de dato	Estrategia
Pagos	Strong Consistency
Inventario	Strong Consistency
Órdenes	Strong Consistency
Catálogo	Eventual Consistency
Reviews	Eventual Consistency
Recomendaciones	Eventual Consistency
Analytics	Eventual Consistency

Tabla 21. Estrategias de consistencia para flujo de sincronización.

Referencias

- Database System Concepts. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). [McGraw-Hill Education](#)
- Fundamentals of Database Systems. Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson Education
- CAP theorem. Brewer, E. A. (2000). *Towards robust distributed systems*. Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing (PODC), 7. <https://doi.org/10.1145/343477.343502>
- Designing Data-Intensive Applications. Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. [O'Reilly Media](#)
- Brazilian E-Commerce Public Dataset by Olist. Olist. (2018). *Brazilian E-Commerce Public Dataset by Olist*. Kaggle. [Kaggle Dataset](#)